

SAPIENZA
UNIVERSITÀ DI ROMA

"Sapienza" University of Rome
Faculty of Information Engineering,
Informatics and Statistics
Department of Computer Science

FDS

Fundamentals of Data Science

Author
Matteo

October 11, 2026

Contents

1 Introduction **1**

1.1 What is Data Science? 1

1.2 The Data Science lifecycle 1

1.3 Artificial Intelligence and Machine Learning 2

1.4 Supervised models and prediction tasks 3

2 Data Representation **4**

2.1 The Iris dataset 4

2.2 Features and targets 5

2.3 Data types 5

2.4 Encoding categorical features 6

2.5 Labels 7

2.6 Dataset notation 8

2.7 Data generation and the i.i.d. assumption 9

2.8 Data points as vectors 10

2.9 Spanning sets 11

2.10 Basis 12

2.11 Standard basis 14

2.12 Coordinates in an orthonormal basis 17

2.13 Nonstandard bases 19

2.14 Finding the weights by least squares 20

2.15 Using fewer directions: subspaces 24

2.16 Projection onto a subspace 25

2.17 Orthonormal directions in a subspace 27

3 Principal Component Analysis **29**

3.1 Linear transformations 29

3.2 Changing coordinates and projecting 31

3.3 Measuring reconstruction error 32

3.4 Learning the basis from the data 33

1 Introduction

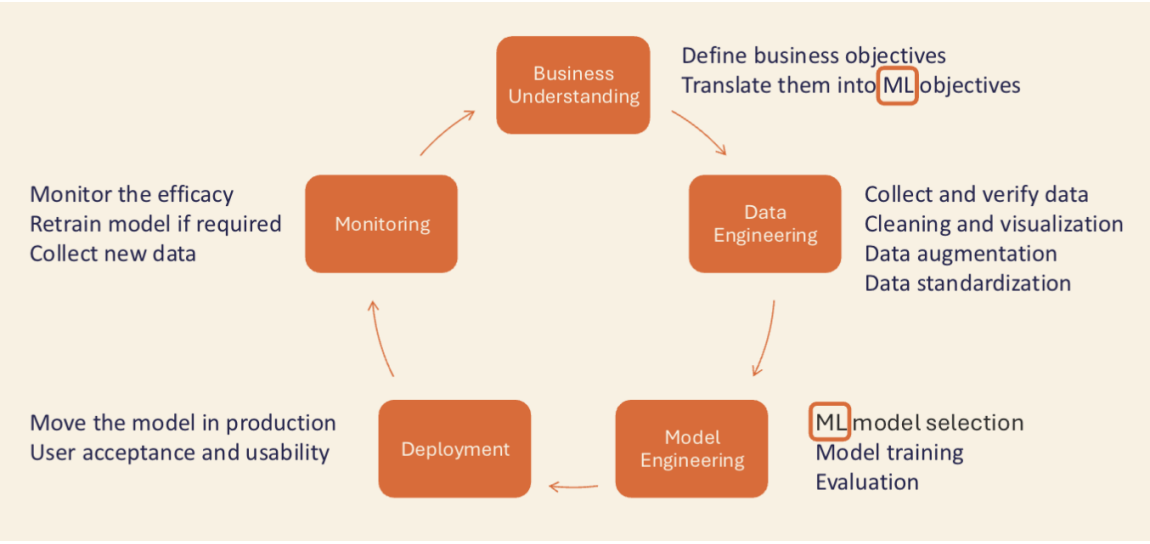
1.1 What is Data Science?

Definition 1.1: Data Science

Data Science is an interdisciplinary field that extracts knowledge from data to support decisions. It combines statistics, mathematics, computer science, and domain expertise to collect, clean, manage, and analyse data.

1.2 The Data Science lifecycle

A project moves through business understanding, data engineering, model engineering, deployment, and monitoring. This cycle can repeat when new data or retraining is needed.



The Data Science lifecycle. Source: Spinelli, Lecture 00.

1.3 Artificial Intelligence and Machine Learning

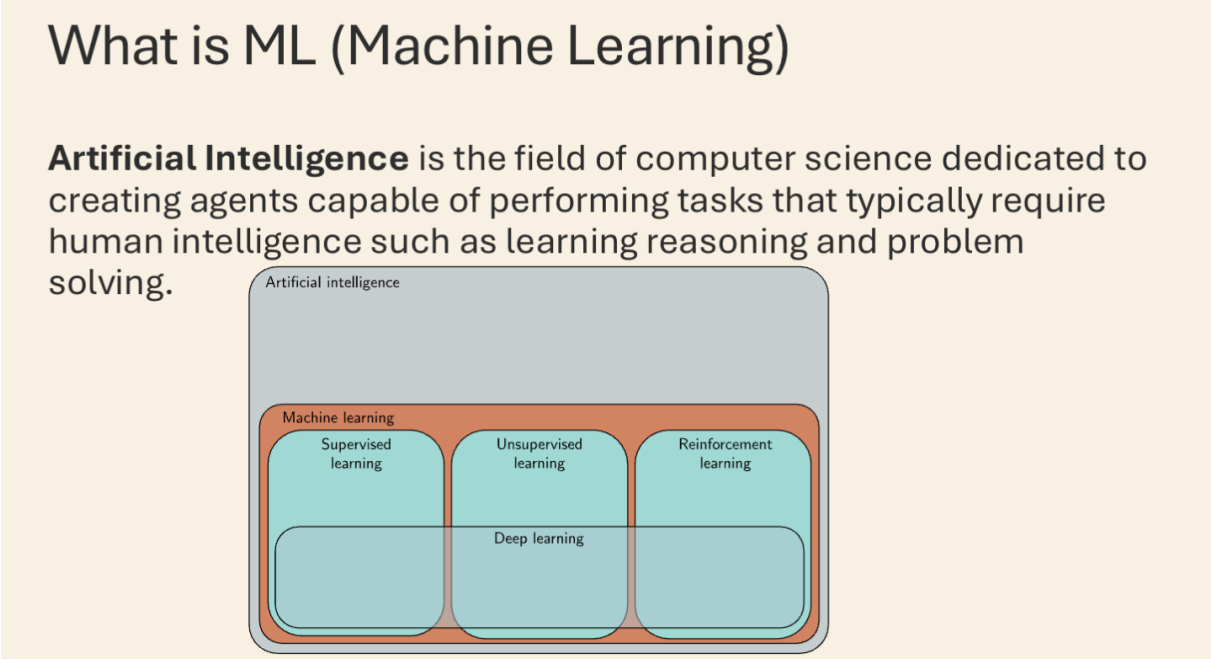
Artificial Intelligence (AI) studies how to build systems that perform tasks requiring abilities such as reasoning, learning, and problem solving.

Definition 1.2: Machine Learning

Machine Learning (ML) studies algorithms whose performance P on a task T improves through experience E. A learning problem therefore specifies the task, the performance measure, and the available experience.

The main learning settings are:

- **Supervised learning:** learn from inputs paired with known desired outputs to predict the output for new inputs.
- **Unsupervised learning:** discover structure in data without given target outputs.
- **Reinforcement learning:** learn how to act through interaction with an environment and feedback in the form of rewards.



AI, ML, and learning settings. Source: Spinelli, Lecture 00.

Deep learning uses neural networks with multiple layers and can be applied in each of these settings.

1.4 Supervised models and prediction tasks

Definition 1.3: Supervised learning model

A supervised learning model is a function $f : \mathcal{X} \to \mathcal{Y}$ that maps an input x to a predicted output $\hat{y} = f(x)$. Here, $\mathcal{X}$ is the input space and $\mathcal{Y}$ is the output space. Training means choosing or adjusting f using examples of inputs paired with their known outputs.

Definition 1.4: Regression

Regression is a supervised learning task whose target is a real-valued number or vector. For example, predicting a person's height from their age is a regression task.

Definition 1.5: Classification

Classification is a supervised learning task whose target is a category from a finite set of classes. For example, predicting whether a flower is setosa, versicolor, or virginica is a classification task.

2

Data Representation

Start of lecture 2 October 2026 (Spinelli)

2.1 The Iris dataset

The Iris dataset contains 150 flowers, divided into three species: 50 setosa, 50 versicolor, and 50 virginica. The goal is to predict a flower's species from its measurements.

Example 1: Iris as a supervised classification task

Each **observation** is one flower, described by four **features**, all measured in centimetres:

- sepal length and sepal width;
- petal length and petal width.

The **target** is the flower's species. This is supervised learning because the species is known for the training examples, and classification because the target is a category.

2.2 Features and targets

In a tabular dataset, each row describes one observation. The feature columns contain its input measurements, while the target column contains the output we want to predict.

For the first flower shown in the lecture's table,

$$x = (4.3,\ 3.0,\ 1.1,\ 0.1), \qquad y = \text{setosa}.$$

The entries of x are, in order, sepal length, sepal width, petal length, and petal width, all in centimetres.

During training, both x and y are known. For a new input x, the model produces a prediction $\hat{y} = f(x)$. The observed target y and the predicted target $\hat{y}$ may differ.

2.3 Data types

Features and targets can have different types:

Type	**Values**	**Example**
Real-valued	$\mathbb{R}$	Petal length
Integer-valued	$\mathbb{Z}$	Number of purchases
Categorical	$\{C_1, \ldots, C_g\}$	Flower species
Binary	$\{0, 1\}$	Spam or non-spam

Predicting a numerical quantity is a regression task; predicting a category is a classification task. In Iris, the four features are numerical measurements and the target has three categories.

Assigning codes $0, 1, 2$ to the species does not turn them into numerical quantities: the codes identify categories, so the task remains classification.

2.4 Encoding categorical features

How can we give a colour to a model that works with numbers? Many learning algorithms require numerical features, so we need an *encoding*: a way to represent a category using numbers.

We could assign the codes red = 0, green = 1, and blue = 2. But a model that treats these codes as quantities could interpret blue as greater than green, or as twice as far from red as green is. Colours have no such order, and these numerical distances have no meaning here.

One-hot encoding replaces the colour column with three yes/no questions: "Is it red?", "Is it green?", and "Is it blue?". We use 1 for yes and 0 for no.

Example 2: Encoding a colour feature

Colour	Is it red?	Is it green?	Is it blue?
Red	1	0	0
Green	0	1	0
Blue	0	0	1

For example, a green product is represented by $(0, 1, 0)^\top$. Exactly one entry is 1 and all the others are 0: this is why the encoding is called *one-hot*.

A model predicting a product's price can then learn a separate effect for each colour, without assuming that blue is "twice" green. The cost is more columns: one feature with k categories becomes k numerical columns. We can now write this idea formally.

Definition 2.1: One-hot encoding

For a feature with k mutually exclusive categories, one-hot encoding represents category C_i by a vector $e_i \in \{0, 1\}^k$ whose i-th entry is 1 and whose other entries are 0:

$$(e_i)_j = \begin{cases} 1, & j = i, \\ 0, & j \neq i. \end{cases}$$

Each observation belongs to exactly one category, so its encoded vector has exactly one nonzero entry.

When categories have a natural order, an *ordinal encoding* can preserve it. For example, low, medium, and high can be coded as $0, 1, 2$. This expresses their order; it does not establish that the gaps between successive categories are equal.

2.5 Labels

Definition 2.2: Labels and labeled data

A *label* is an observed target value. In *labeled data*, the input and its target are both available.
In *unlabeled data*, the input is available but its target is unknown or unrecorded.

In Iris, a flower with measurements and a known species is labeled; the same measurements without the species are unlabeled. A label can also be numerical, as in a regression task.

2.6 Dataset notation

Let n be the number of observations and p the number of numerical features after any encoding. Write $\mathcal{X} \subseteq \mathbb{R}^p$ for the input space and $\mathcal{Y}$ for the target space.

Definition 2.3: Supervised dataset

A labeled dataset is an indexed collection of input–target pairs,

$$D = \big((x^{(1)}, y^{(1)}), \ldots, (x^{(n)}, y^{(n)})\big) \in (\mathcal{X} \times \mathcal{Y})^n.$$

The i-th observation has an input column vector

$$x^{(i)} = (x_1^{(i)}, \ldots, x_p^{(i)})^\top \in \mathbb{R}^p$$

and a target $y^{(i)} \in \mathcal{Y}$.

The superscript (i) indexes an observation; the subscript j indexes a feature. Across all observations, the j-th feature column is

$$\mathbf{x}_j = (x_j^{(1)}, \ldots, x_j^{(n)})^\top \in \mathbb{R}^n.$$

Thus an input vector contains p features of one observation, while a feature column contains n values of one feature.

Example 3: A small supervised dataset

Suppose we predict house prices from area and room count. Here is a toy dataset:

House i	Area (m^2)	Rooms	Price (EUR)
1	50	2	150 000
2	80	3	240 000
3	100	4	300 000

Here $n = 3$ houses and $p = 2$ input features. The price is the target, so it is not counted in p. For the first house,

$$x^{(1)} = (50, 2)^\top, \qquad y^{(1)} = 150\,000.$$

D collects the three input–target pairs. Here $x_2^{(1)} = 2$ is the first house's room count. The rooms column is

$$\mathbf{x}_2 = (2, 3, 4)^\top.$$

The vector $x^{(1)}$ describes one house using both features; the vector $\mathbf{x}_2$ contains one feature for all three houses.

2.7 Data generation and the i.i.d. assumption

We model the observations as realizations of random pairs (X, Y) with an unknown joint probability distribution P_{XY} on $\mathcal{X} \times \mathcal{Y}$. Capital letters denote random variables; lowercase letters denote their observed values. Supervised learning uses these observations to learn a predictive relationship between input and target.

Definition 2.4: I.i.d. observations

The observations are *i.i.d.* when their random pairs $(X^{(1)}, Y^{(1)}), \ldots, (X^{(n)}, Y^{(n)})$ are mutually independent and all have the same distribution P_{XY}.

Independence is between observations; it does not require X and Y within one observation to be independent. The i.i.d. assumption can fail, for example when successive observations form a time series.

2.8 Data points as vectors

We now focus on the inputs $x^{(i)} \in \mathbb{R}^p$. This space supports vector addition and multiplication by real numbers, called *scalars*. These operations act coordinate by coordinate:

$$u + v = (u_1 + v_1, \ldots, u_p + v_p)^\top, \qquad au = (au_1, \ldots, au_p)^\top.$$

An input can be drawn as a point with those coordinates or as a *position vector*: an arrow from the origin to that point.

2.8.1 Mean-centering

Definition 2.5: Mean-centered data

The sample mean input vector and the centered observations are

$$\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x^{(i)}, \qquad \widetilde{x}^{(i)} = x^{(i)} - \bar{x}.$$

The mean of the centered inputs is zero: $\frac{1}{n} \sum_{i=1}^{n} \widetilde{x}^{(i)} = 0$.

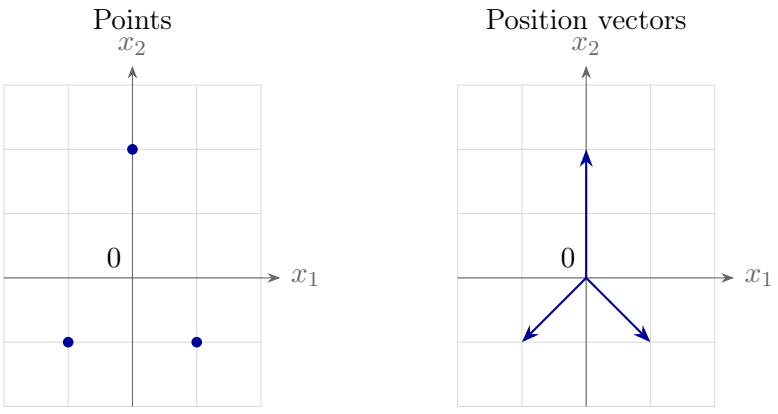
Centering subtracts each feature's sample mean from that feature. It is reversible if we retain $\bar{x}$, since $x^{(i)} = \widetilde{x}^{(i)} + \bar{x}$.

Example 4: Centering three points

For the inputs $(1, 1)^\top$, $(3, 1)^\top$, and $(2, 4)^\top$,

$$\bar{x} = \left(\frac{1+3+2}{3}, \frac{1+1+4}{3} \right)^\top = (2, 2)^\top.$$

The centered inputs are $(-1, -1)^\top$, $(1, -1)^\top$, and $(0, 2)^\top$. Their sum is $(0, 0)^\top$.



The same centered observations, drawn as points and as arrows.

2.9 Spanning sets

Definition 2.6: Span and spanning set

The *span* of vectors $c_1, \ldots, c_K$ is the set of all their linear combinations:

$$\operatorname{span}\{c_1, \ldots, c_K\} = \left\{ \sum_{j=1}^{K} a_j c_j : a_1, \ldots, a_K \in \mathbb{R} \right\}.$$

These vectors form a *spanning set* of a vector space V when their span equals V: every vector in V can be expressed using them.

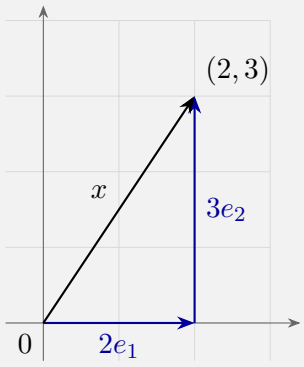
Example 5: Spanning the plane

The vectors

$$e_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad e_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

span $\mathbb{R}^2$, because every $(a, b)^\top$ can be written as $ae_1 + be_2$. For example,

$$\begin{pmatrix} 2 \\ 3 \end{pmatrix} = 2e_1 + 3e_2.$$



Using only e_1 produces vectors $(a, 0)^\top$, so it spans the horizontal axis rather than the whole plane.

2.10 Basis

Definition 2.7: Linear independence

Vectors $c_1, \ldots, c_K \in \mathbb{R}^p$ are *linearly independent* if the only scalars $a_1, \ldots, a_K \in \mathbb{R}$ satisfying

$$a_1 c_1 + \cdots + a_K c_K = 0$$

are $a_1 = \cdots = a_K = 0$.

Definition 2.8: Basis

A *basis* of a vector space is a linearly independent spanning set. Equivalently, it is a minimal spanning set: removing any of its vectors destroys the spanning property.

Example 6: A spanning set with a redundant vector

The pair $\{e_1, e_2\}$ is a basis of $\mathbb{R}^2$. Adding $c_3 = e_1 + e_2$ still gives a spanning set, but it is not a basis, since

$$e_1 + e_2 - c_3 = 0$$

is a linear combination with nonzero coefficients. The third vector is redundant because the first two already generate it.

2.10.1 Representing observations in a basis

A basis of $\mathbb{R}^p$ contains exactly p vectors, each in $\mathbb{R}^p$. For a basis $c_1, \ldots, c_p$, every input observation has a unique representation

$$x^{(i)} = \sum_{j=1}^{p} w_j^{(i)} c_j, \qquad i = 1, \ldots, n.$$

The coefficients $w_j^{(i)}$ are the *weights*, or coordinates, of that observation in the chosen basis. The vector $w^{(i)} = (w_1^{(i)}, \ldots, w_p^{(i)})^\top$ records its representation; combining the basis vectors with these weights reconstructs $x^{(i)}$.

Example 7: Coordinates in a basis

Use the same basis for both observations:

$$c_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad c_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

The two points have different weights:

$$x^{(1)} = \begin{pmatrix} 2 \\ 3 \end{pmatrix} = 2c_1 + 3c_2,$$

$$x^{(2)} = \begin{pmatrix} 4 \\ 1 \end{pmatrix} = 4c_1 + c_2.$$

Observation	**Weight of c_1**	**Weight of c_2**
First	$w_1^{(1)} = 2$	$w_2^{(1)} = 3$
Second	$w_1^{(2)} = 4$	$w_2^{(2)} = 1$

For example, $w_2^{(1)} = 3$ means that the first observation uses three times the second basis vector. The superscript (i) identifies the observation; the subscript j identifies the basis vector.

2.11 Standard basis

Definition 2.9: Standard basis

The *standard basis* of $\mathbb{R}^p$ consists of the vectors $c_1, \ldots, c_p$, where c_j has a 1 in position j and zeros everywhere else. In $\mathbb{R}^2$,

$$c_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad c_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

Each vector points along one coordinate axis. Consequently, the weights in this basis are the entries of the observation itself:

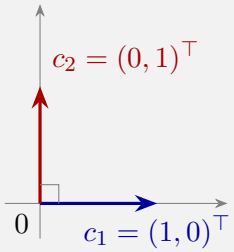
$$x^{(i)} = x_1^{(i)} c_1 + \cdots + x_p^{(i)} c_p \qquad \Longrightarrow \qquad w^{(i)} = x^{(i)}.$$

Example 8: Reading the weights from the coordinates

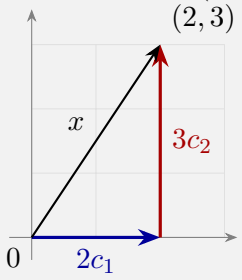
For $x = (2, 3)^\top$, move two units along the first axis and three along the second:

$$x = 2c_1 + 3c_2, \qquad w = \begin{pmatrix} 2 \\ 3 \end{pmatrix} = x.$$

Unit basis vectors



Representing $x = (2, 3)^\top$



The blue arrow contributes the first coordinate; the red arrow contributes the second. Their sum is the black arrow.

2.11.1 Orthogonality, unit length, and the basis matrix

The standard basis has two properties:

- **Different vectors are perpendicular:** $c_j^\top c_k = 0$ for $j \neq k$.
- **Each vector has unit length:** $\|c_j\|_2^2 = c_j^\top c_j = 1$.

In $\mathbb{R}^2$, these follow directly from

$$c_1^\top c_2 = 1 \cdot 0 + 0 \cdot 1 = 0, \qquad \|c_1\|_2^2 = 1^2 + 0^2 = 1,$$

and similarly $\|c_2\|_2^2 = 1$.

Definition 2.10: Orthonormal basis

A basis is *orthonormal* if its vectors are mutually perpendicular and each has length 1.

How can we check whether a basis is orthonormal? We need to check the length of every vector and whether each pair of different vectors is perpendicular. With many vectors, drawing arrows is impractical. We can collect all these checks in one matrix.

First, place the basis vectors in the columns of C:

$$C = [\, c_1 \ \cdots \ c_p \,] \in \mathbb{R}^{p \times p}.$$

Now compute $C^\top C$. Each entry is a dot product between two columns of C: the entry in row j, column k is $c_j^\top c_k$. We read the result as a table of checks:

Position	What it tells us	Required value
On the diagonal	A vector's length squared	1
Off the diagonal	The dot product of two different vectors	0

If the diagonal contains only 1s and every other entry is 0, both checks pass: the basis is orthonormal. That result is the identity matrix I_p, so the complete test is

$$\boxed{C^\top C = I_p.}$$

This formula is a compact way to check the lengths and perpendicularity of all the basis vectors at once. The standard basis passes the test.

Example 9: Using the orthonormality test

Consider the basis

$$c_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad c_2 = \begin{pmatrix} 0.6 \\ 0.8 \end{pmatrix}, \qquad C = \begin{pmatrix} 1 & 0.6 \\ 0 & 0.8 \end{pmatrix}.$$

Both vectors have length 1, since $1^2 + 0^2 = 1$ and $0.6^2 + 0.8^2 = 1$. Do they also pass the perpendicularity check?

$$C^\top C = \begin{pmatrix} 1 & 0 \\ 0.6 & 0.8 \end{pmatrix} \begin{pmatrix} 1 & 0.6 \\ 0 & 0.8 \end{pmatrix} = \begin{pmatrix} 1 & 0.6 \\ 0.6 & 1 \end{pmatrix}.$$

The diagonal 1s confirm the unit lengths. The off-diagonal entries are 0.6, so the vectors are not perpendicular. Therefore $C^\top C \neq I_2$: this basis is not orthonormal, even though both vectors have length 1.

In the standard basis, the weights equal the entries of the observation: $w = x$. For a different orthonormal basis, the same observation can have different weights. The test $C^\top C = I_p$ checks the geometry of the basis; the chosen basis determines the weights.

2.12 Coordinates in an orthonormal basis

We now know how to check a basis. How do we find the weights needed to represent a point in that basis? For an orthonormal basis, a dot product with each basis vector gives the answer.

We know the observation x and choose the basis vectors $c_1, \ldots, c_p$. We want to find their weights $w_1, \ldots, w_p$ so that their weighted sum gives x. Put the basis vectors in the columns of C and the weights in w. Matrix multiplication performs that sum:

$$Cw = w_1 c_1 + \cdots + w_p c_p = x.$$

Thus $x = Cw$ is the compact form of a linear combination. Multiplying both sides by $C^\top$ gives

$$C^\top x = C^\top C w = I_p w = w.$$

Therefore the coordinates are

$$\boxed{w = C^\top x, \qquad w_j = c_j^\top x.}$$

The weight w_j tells us how far to move along the unit vector c_j. A positive weight follows its direction; a negative weight goes in the opposite direction. Perpendicular basis vectors do not contribute to this dot product, so it picks out just the weight we want.

For every observation $x^{(i)}$, the same calculation gives $w^{(i)} = C^\top x^{(i)}$. With a complete orthonormal basis, combining these weights with the basis vectors gives the original observation:

$$Cw^{(i)} = CC^\top x^{(i)} = x^{(i)}.$$

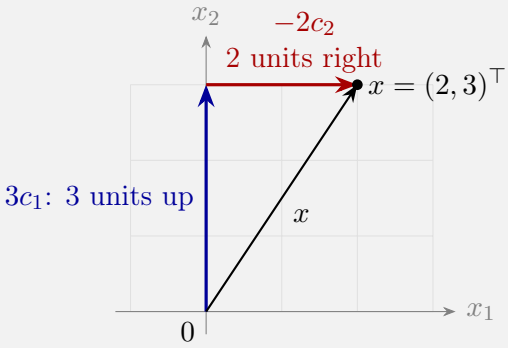
Here C is square and its columns form a basis of all of $\mathbb{R}^p$, so $CC^\top = I_p$ as well as $C^\top C = I_p$.

Example 10: Finding the weights of the point $(2, 3)$

Our observation is $x = (2, 3)^\top$: two units to the right and three units up. We want to describe this same point using two different unit directions:

$$c_1 = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \quad \text{(up)}, \qquad c_2 = \begin{pmatrix} -1 \\ 0 \end{pmatrix} \quad \text{(left)}.$$

These vectors are perpendicular and have length 1, so they form an orthonormal basis. We seek weights w_1, w_2 for which $x = w_1 c_1 + w_2 c_2$.



Read the two moves in the drawing:

- Along c_1, we need to go up three units, so $w_1 = 3$.
- c_2 points left, but we need to go right two units. We move against its direction, so $w_2 = -2$.

The dot-product rule gives exactly these weights:

$$w_1 = c_1^\top x = 0 \cdot 2 + 1 \cdot 3 = 3,$$
$$w_2 = c_2^\top x = (-1) \cdot 2 + 0 \cdot 3 = -2.$$

Thus $w = (3, -2)^\top$. To check these weights, add their contributions:

$$Cw = 3c_1 - 2c_2 = \begin{pmatrix} 0 \\ 3 \end{pmatrix} + \begin{pmatrix} 2 \\ 0 \end{pmatrix} = \begin{pmatrix} 2 \\ 3 \end{pmatrix} = x.$$

The sum gives our original observation. Its weights are $(2, 3)$ in the standard basis and $(3, -2)$ in the chosen basis. Both sets of weights describe the same point using their respective basis vectors.

2.13 Nonstandard bases

The standard basis is convenient, but we can choose other directions. They must be linearly independent and span the space; they do not have to be perpendicular or have unit length.

Consider the basis from the slides:

$$c_1 = \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \qquad c_2 = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \qquad C = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}.$$

The vectors are independent, but $c_1^\top c_2 = 4$ and each has length $\sqrt{5}$. This basis is not orthonormal.

To find the weights, we must still solve $Cw = x$.

Example 11: Representing $(5, 5)$ in a different basis

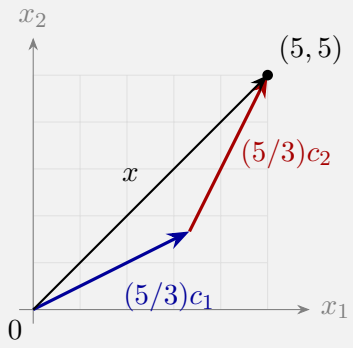
For $x = (5, 5)^\top$, the equation $w_1 c_1 + w_2 c_2 = x$ becomes

$$\begin{cases} 2w_1 + w_2 = 5, \\ w_1 + 2w_2 = 5. \end{cases}$$

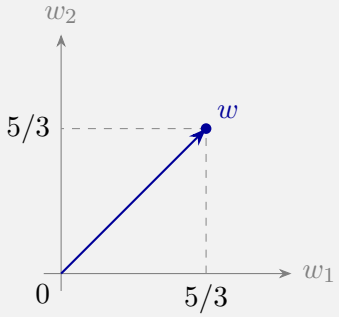
Solving the system gives $w_1 = w_2$, so

$$w_1 = w_2 = \frac{5}{3}, \qquad x = \frac{5}{3}c_1 + \frac{5}{3}c_2.$$

Original coordinates



New coordinates



On the left, the two contributions add up to the original point. On the right, we record their weights: $w = (5/3, 5/3)^\top$.

2.13.1 Encoding and decoding

Here we use the same basis as in the previous example. Its two vectors are the columns of the matrix C:

$$c_1 = \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \qquad c_2 = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \qquad C = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}.$$

The vector $w = (w_1, w_2)^\top$ stores the two weights: how much of each basis vector we need to add. Multiplying C by w performs this sum:

$$Cw = w_1 c_1 + w_2 c_2.$$

Thus Cw is a point expressed in the original coordinates.

Encoding means starting from a point x and finding the weights w such that $Cw = x$. *Decoding* means starting from the weights w and computing Cw to obtain the point in its original coordinates.

For example, keep this same matrix C and choose the weights $w = (-2, 1)^\top$. Decoding gives

$$Cw = -2c_1 + c_2 = -2 \begin{pmatrix} 2 \\ 1 \end{pmatrix} + \begin{pmatrix} 1 \\ 2 \end{pmatrix} = \begin{pmatrix} -3 \\ 0 \end{pmatrix}.$$

These weights describe the point $x = (-3, 0)^\top$. The weight -2 means taking twice c_1 in the opposite direction; the weight 1 means adding one copy of c_2.

Because these vectors form a basis, each point has exactly one pair of weights. Encoding a point and then decoding its weights returns that same point.

2.14 Finding the weights by least squares

We know the point x and the basis matrix C. We want to find the weights w with which to combine the columns of C. Each choice of weights produces a point Cw. Our question is: *which weights make this point as close as possible to x?*

Start with the basis and point from the previous example:

$$C = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}, \qquad x = \begin{pmatrix} 5 \\ 5 \end{pmatrix}.$$

Suppose we try the weights $w = (1, 1)^\top$. Adding one copy of each basis vector gives

$$Cw = \begin{pmatrix} 3 \\ 3 \end{pmatrix}, \qquad Cw - x = \begin{pmatrix} -2 \\ -2 \end{pmatrix}.$$

We wanted $(5, 5)$, but obtained $(3, 3)$. The difference $Cw - x$ is the *reconstruction error*: how far off we are on each coordinate. To measure it with a single number, square each coordinate error and add the results. For these weights, the result is

$$(-2)^2 + (-2)^2 = 8.$$

Squaring makes each contribution nonnegative, so errors on different coordinates cannot cancel each other.

For a point with p coordinates, we call this sum $h(w)$:

$$h(w) = \|Cw - x\|_2^2 = \sum_{j=1}^{p} \big((Cw)_j - x_j\big)^2.$$

Here $(Cw)_j$ is the jth coordinate of the reconstructed point and x_j is the corresponding coordinate of the original point. *Least squares* means choosing the weights that make $h(w)$ as small as possible.

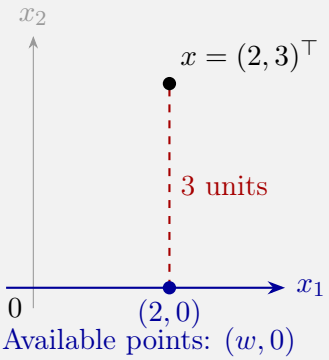
With a complete basis, we can reconstruct x exactly, so the minimum error is zero. We will obtain the same weights as by solving $Cw = x$. This formulation will also let us find the best approximation when we keep fewer directions and exact reconstruction is impossible.

Example 12: Approximating a point with one direction

Take the point $x = (2, 3)^\top$: two units to the right and three units up. Suppose we have only one direction: the horizontal vector $c_1 = (1, 0)^\top$. The matrix C now has one column, and the weight w is a single number:

$$C = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad Cw = \begin{pmatrix} w \\ 0 \end{pmatrix}.$$

Every point we can reconstruct lies on the horizontal axis: its second coordinate is always zero.



The point x is three units above the horizontal axis, so we cannot reach it exactly. The closest available point is $(2, 0)$, directly below x, obtained by choosing $w = 2$.

The squared reconstruction error is

$$h(w) = (w - 2)^2 + (0 - 3)^2 = (w - 2)^2 + 9.$$

Choosing $w = 2$ makes the first term zero. The minimum squared error is therefore 9.

We can also find the best weight by setting the derivative of h to zero:

$$h'(w) = 2(w - 2) = 0 \quad \Longrightarrow \quad w = 2.$$

2.14.1 The normal equations

In the previous example, we found the best value of a single weight w by solving $h'(w) = 0$. We now generalise this procedure from one weight to several weights, finding their best values together. This will give one equation for each weight.

If C has K columns, we need K weights $w_1, \ldots, w_K$. With a complete basis, $K = p$; with fewer directions, $K < p$.

To find the weights that minimise h, we compute a derivative with respect to each weight. The derivative $\partial h/\partial w_j$ describes how the error changes when we change w_j slightly while keeping the other weights fixed.

The vector collecting all these derivatives is called the *gradient*, written $\nabla_w h$:

$$\nabla_w h(w) = \left(\frac{\partial h}{\partial w_1}, \ldots, \frac{\partial h}{\partial w_K} \right)^\top.$$

At a minimum, all these derivatives are zero. For this quadratic function, when the columns of C are linearly independent, setting the gradient to zero finds the unique minimum.

Recall the squared reconstruction error:

$$h(w) = \|Cw - x\|_2^2 = \sum_{j=1}^{p} \big((Cw)_j - x_j \big)^2.$$

Differentiating this function with respect to the weights gives

$$\nabla_w h(w) = 2C^\top (Cw - x) = 0.$$

Divide by 2 and expand the product:

$$C^\top (Cw - x) = 0 \quad \Longrightarrow \quad C^\top Cw - C^\top x = 0.$$

Moving the second term to the right gives a linear system called the *normal equations*:

Proposition 2.1: Normal equations

For a fixed matrix C with linearly independent columns, the unique weights that minimise $\|Cw - x\|_2^2$ satisfy

$$\boxed{C^\top Cw = C^\top x.}$$

The matrix $C^\top C$ and the vector $C^\top x$ are known, because we know C and x. We solve this system for the unknown weights w. The matrix $C^\top C$ accounts for the lengths of the chosen vectors and the dot products between them. If the columns are orthonormal, it equals the identity and the equations reduce to $w = C^\top x$.

Example 13: Solving the normal equations

For our basis and point, the known quantities are

$$C = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}, \qquad x = \begin{pmatrix} 5 \\ 5 \end{pmatrix},$$

$$C^\top C = \begin{pmatrix} 5 & 4 \\ 4 & 5 \end{pmatrix}, \quad C^\top x = \begin{pmatrix} 15 \\ 15 \end{pmatrix}.$$

The matrix equation $C^\top C w = C^\top x$ becomes

$$\begin{cases} 5w_1 + 4w_2 = 15, \\ 4w_1 + 5w_2 = 15. \end{cases}$$

Solving gives $w_1 = w_2 = 5/3$. With these weights, $Cw = (5, 5)^\top = x$, so $h(w) = 0$: we reconstruct the point exactly.

Taking $C^\top x$ directly as the weights would instead give $(15, 15)^\top$ and reconstruct $(45, 45)^\top$. The factor $C^\top C$ is essential for this basis.

2.14.2 Encoding the whole dataset

For a dataset of n observations, repeat this calculation for each point $x^{(i)}$. Keep the same basis matrix C, but find a separate weight vector $w^{(i)}$ for each point. The average reconstruction error is

$$\frac{1}{n} \sum_{i=1}^{n} \|Cw^{(i)} - x^{(i)}\|_2^2.$$

Each term depends only on that observation's weights, so we can minimise the terms separately by solving

$$C^\top C w^{(i)} = C^\top x^{(i)}, \qquad i = 1, \ldots, n.$$

The basis matrix stays the same; the observation and its weights change.

2.15 Using fewer directions: subspaces

A basis of $\mathbb{R}^p$ has exactly p linearly independent vectors. Using more than p vectors can still give a spanning set, but those vectors cannot all be independent. Using fewer than p independent vectors gives a different possibility: we can represent only part of the original space.

Let $K < p$ be the number of directions we keep. The vectors $c_1, \ldots, c_K$ still have p entries; we now place them in a rectangular matrix

$$C = [\, c_1 \ \cdots \ c_K \,] \in \mathbb{R}^{p \times K}.$$

Combining its columns gives a *subspace* of $\mathbb{R}^p$.

Definition 2.11: Subspace spanned by the columns

The space generated by $c_1, \ldots, c_K$ is the set of all their linear combinations:

$$S = \operatorname{span}\{c_1, \ldots, c_K\} = \{Cw : w \in \mathbb{R}^K\}.$$

If the columns are linearly independent, S has dimension K and they form a basis of S. This set contains the origin and is closed under vector addition and multiplication by scalars.

In two dimensions, keeping one direction gives a line through the origin. In three dimensions, keeping two independent directions gives a plane through the origin. A subspace of dimension $p - 1$ is called a *hyperplane*.

Example 14: A line in a two-dimensional space

Keep only $c_1 = (1, 1)^\top$. Then

$$Cw = w \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} w \\ w \end{pmatrix}.$$

Every combination has equal coordinates: the available points lie on the line $x_2 = x_1$. We can represent $(3, 3)^\top$ exactly with $w = 3$. But no value of w gives $(2, 0)^\top$: its first coordinate would require $w = 2$ and its second would require $w = 0$.

We now store just K weights for an observation that originally had p coordinates. This is dimensionality reduction. It can be useful when the observations lie on, or close to, a smaller subspace. An observation outside that subspace cannot be recovered exactly; next we find its closest available approximation.

2.16 Projection onto a subspace

Suppose the chosen directions cannot reach x. We still want the point on their subspace that is closest to x. For a line, imagine dropping a perpendicular from x to the line: its endpoint is the approximation we keep.

Definition 2.12: Orthogonal projection

Let the columns of $C \in \mathbb{R}^{p\times K}$ be linearly independent and span S. The orthogonal projection of x onto S is

$$\hat{x} = Cw_*, \qquad w_* = \arg\min_{w\in\mathbb{R}^K} \|Cw - x\|_2^2.$$

Thus $\hat{x}$ is the point of S with the smallest Euclidean distance from x; w_* contains its coordinates in the chosen basis of S.

The calculation is the same as before:

$$C^\top C w_* = C^\top x.$$

This time the K weights describe the closest point, which may differ from x. Since the columns are independent, $C^\top C$ is invertible, so we can also write the solution as

$$w_* = (C^\top C)^{-1} C^\top x, \qquad \hat{x} = C(C^\top C)^{-1} C^\top x.$$

Example 15: The closest point on the line $x_2 = x_1$

For $C = (1, 1)^\top$ and $x = (2, 0)^\top$, the normal equations give

$$C^\top C = 2, \qquad C^\top x = 2, \qquad 2w_* = 2.$$

Hence $w_* = 1$ and

$$\hat{x} = Cw_* = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

We cannot reach $(2, 0)$ on this line, but $(1, 1)$ is the closest point. The error left over is $x - \hat{x} = (1, -1)^\top$, whose squared length is $1^2 + (-1)^2 = 2$.

2.16.1 Why the projection is perpendicular

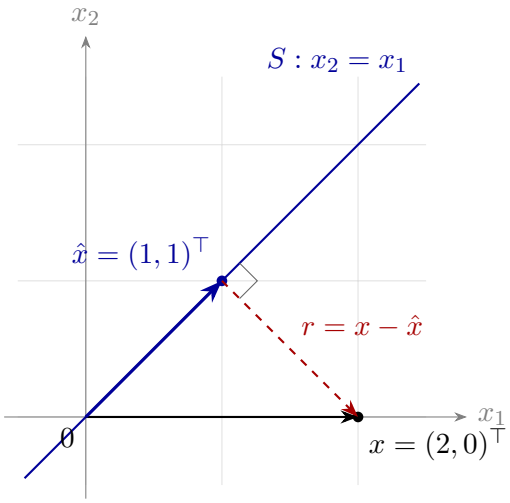
Call the part left over the *residual*, $r = x - \hat{x}$. The normal equations say

$$C^\top r = C^\top (x - Cw_*) = 0.$$

Each entry is $c_j^\top r = 0$: the residual is perpendicular to every chosen direction, and therefore to their whole subspace. Moving the approximation within that subspace cannot recover this missing part. In our example, $(1, 1)^\top (1, -1) = 1 - 1 = 0$.

2.16.2 Seeing the projection and its encoding

The blue line contains all the points we can represent with $(1, 1)^\top$. The dashed segment shows the residual from the projected point to x.



The residual joins the projection to the original point at a right angle. It measures what is lost when we keep only this line.

With a fixed C, the two steps are:

Step	What we do
Encoding	Find the K weights w_* that best represent x.
Decoding	Compute $\hat{x} = Cw_*$, returning to p coordinates.

For the example, the whole process is

$$\underbrace{\begin{pmatrix} 2 \\ 0 \end{pmatrix}}_{\text{original: 2 coordinates}} \xrightarrow{\text{encode}} \underbrace{1}_{\text{1 weight}} \xrightarrow{\text{decode}} \underbrace{\begin{pmatrix} 1 \\ 1 \end{pmatrix}}_{\text{reconstructed: 2 coordinates}} .$$

Decoding restores the number of coordinates, but it cannot restore the discarded residual. If x already belongs to S, the residual is zero and the reconstruction is exact.

2.17 Orthonormal directions in a subspace

We can make projection easier by choosing perpendicular unit vectors as the basis of the subspace. There are only K such vectors, so C still has p rows and K columns. Their lengths and dot products give

$$C^\top C = I_K.$$

The normal equations then give the same useful shortcut as before:

$$\boxed{w_* = C^\top x, \qquad \hat{x} = Cw_* = CC^\top x.}$$

Each dot product measures how much of x lies along a chosen direction. Adding those contributions gives the projection.

Example 16: Normalising the direction of the same line

Our line can also be described by the unit vector

$$c_1 = \frac{1}{\sqrt{2}}\begin{pmatrix}1\\1\end{pmatrix}, \qquad C = c_1.$$

For $x = (2, 0)^\top$, the shortcut gives

$$w_* = c_1^\top x = \sqrt{2}, \qquad \hat{x} = \sqrt{2}c_1 = \begin{pmatrix}1\\1\end{pmatrix}.$$

Previously the direction was $(1, 1)^\top$ and its weight was 1. After making it a unit vector, the weight becomes $\sqrt{2}$. The line and the projected point stay the same.

Here the two matrix products have different sizes and jobs:

$$C^\top C = (1), \qquad CC^\top = \frac{1}{2}\begin{pmatrix}1 & 1\\1 & 1\end{pmatrix}.$$

The first checks the unit length of our one direction. The second projects a two-dimensional point onto the line:

$$CC^\top \begin{pmatrix}2\\0\end{pmatrix} = \begin{pmatrix}1\\1\end{pmatrix}.$$

For $K < p$, $CC^\top$ is a projection matrix, not I_p: it keeps the component in S and removes the perpendicular residual. If instead $K = p$ and the columns form a complete orthonormal basis, $CC^\top = I_p$ and every point is recovered exactly.

2.17.1 From three coordinates to two

The same idea works for a plane in three dimensions. We retain two weights and decode them as a point on that plane.

Example 17: Projection onto the coordinate plane

Choose two perpendicular unit vectors in $\mathbb{R}^3$:

$$c_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \qquad c_2 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \qquad C = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix}.$$

They span the plane $x_3 = 0$. Since $C^\top C = I_2$, for $x = (2, 3, 4)^\top$ the encoding is

$$w_* = C^\top x = \begin{pmatrix} 2 \\ 3 \end{pmatrix}.$$

Decoding gives

$$\hat{x} = Cw_* = \begin{pmatrix} 2 \\ 3 \\ 0 \end{pmatrix}, \qquad r = x - \hat{x} = \begin{pmatrix} 0 \\ 0 \\ 4 \end{pmatrix}.$$

The first two coordinates are preserved. The third is lost because the chosen directions cannot move us out of the plane. The residual is perpendicular to that plane, and its squared length is $4^2 = 16$.

2.17.2 What changes when we reduce the dimension?

For independent columns, the same normal equations work in every case:

$$C^\top C w_* = C^\top x, \qquad \hat{x} = Cw_*.$$

What changes is how many directions we keep and whether they are orthonormal:

Directions	Finding the weights	Reconstruction
Complete basis	Solve the normal equations.	Always $\hat{x} = x$.
Complete, orthonormal	$w_* = C^\top x$.	Always $\hat{x} = x$.
Basis of a subspace	Solve the normal equations.	Closest point in S.
Orthonormal in a subspace	$w_* = C^\top x$.	Closest point in S.

With fewer directions, reconstruction is exact precisely when the observation already lies in their subspace. Otherwise we retain its projection and lose its perpendicular component. The average squared length of these residuals is the dataset's reconstruction error.

Source: Spinelli, Lecture 01 (data_vectors), slides 18–35. The slide deck credits Machine Learning Refined, Chapter 8.

3

Principal Component Analysis

Start of lecture: PCA (Spinelli, Lecture 03)

So far, we have computed weights in a given basis. We now ask how to choose useful directions from the data, using matrix transformations and reconstruction error.

3.1 Linear transformations

A matrix can change a vector's coordinates, stretch a direction, rotate a point, or project it onto a smaller space. Its entries specify how to combine the input coordinates to produce the output coordinates.

Definition 3.1: Linear transformation

For a fixed matrix $A \in \mathbb{R}^{q \times p}$, the map

$$T : \mathbb{R}^p \to \mathbb{R}^q, \qquad T(x) = Ax$$

is a linear transformation. The input x has p coordinates and the output $y = Ax$ has q coordinates. Each row of A forms one output coordinate as a weighted sum of the input coordinates. The map preserves vector addition and multiplication by scalars; in particular, it maps the origin to the origin.

Example 18: Reading what a matrix does

For $x = (2, 1)^\top$, read the multiplication one row at a time:

$$A = \begin{pmatrix} 2 & 0 \\ 0 & 0.5 \end{pmatrix}, \qquad y = Ax = \begin{pmatrix} 2 \cdot 2 + 0 \cdot 1 \\ 0 \cdot 2 + 0.5 \cdot 1 \end{pmatrix} = \begin{pmatrix} 4 \\ 0.5 \end{pmatrix}.$$

The first row doubles the first coordinate; the second row halves the second coordinate. The transformed vector is $y = (4, 0.5)^\top$.

3.1.1 What different matrices do

Apply each of the following matrices to the same input $x = (2, 1)^\top$. The output makes the geometric effect concrete.

Type	A	$y = Ax$	**Effect**
Scaling	$\begin{pmatrix} 2 & 0 \\ 0 & 0.5 \end{pmatrix}$	$\begin{pmatrix} 4 \\ 0.5 \end{pmatrix}$	Double the first coordinate; halve the second.
Rotation	$\begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$	$\begin{pmatrix} -1 \\ 2 \end{pmatrix}$	Rotate the point 90° counterclockwise.
Reflection	$\begin{pmatrix} -1 & 0 \\ 0 & 1 \end{pmatrix}$	$\begin{pmatrix} -2 \\ 1 \end{pmatrix}$	Reflect across the vertical axis.
Shear	$\begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$	$\begin{pmatrix} 3 \\ 1 \end{pmatrix}$	Add the second coordinate to the first.
Projection	$\begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$	$\begin{pmatrix} 2 \\ 0 \end{pmatrix}$	Keep the horizontal component.

For a square matrix, $A^\top A = I_p$ means that its columns are orthonormal. Such a transformation preserves lengths and distances; rotations and reflections are examples. The scaling and shear matrices above change some lengths or angles.

Off-diagonal entries mix input coordinates. Both the rotation and the shear above have such entries: their effect depends on the whole matrix.

3.1.2 How many directions survive?

Definition 3.2: Rank of a matrix

The rank of A is the dimension of the space spanned by its columns: the number of linearly independent output directions they provide.

The projection matrix in the table has rank 1: its outputs all lie on the horizontal axis. Both $(2, 1)^\top$ and $(2, -1)^\top$ become $(2, 0)^\top$, so the output cannot tell us which input we started with.

More generally, if $\text{rank}(A) = K < p$, some input directions are lost and the transformation cannot be inverted for every input. Orthogonal projection is one way to reduce the number of directions.

3.2 Changing coordinates and projecting

The previous lecture used a matrix C whose columns are the chosen directions. Assume they are perpendicular unit vectors. Then $C^\top$ computes the weights, and C combines those weights into a vector in the original space. The number of columns determines what can be kept.

3.2.1 A complete basis keeps every direction

With p basis vectors in $\mathbb{R}^p$, C is square and $C^\top C = CC^\top = I_p$. Encoding and decoding give

$$w = C^\top x, \qquad Cw = CC^\top x = x.$$

We have p new coordinates for a point with p original coordinates. Every point can be reconstructed exactly, and distances between points are preserved by the orthogonal change of coordinates.

3.2.2 Fewer directions keep only a projection

With $K < p$ columns, $C \in \mathbb{R}^{p\times K}$ spans a smaller subspace S. Now

$$C^\top C = I_K, \qquad w = C^\top x \in \mathbb{R}^K, \qquad \hat{x} = Cw = CC^\top x \in \mathbb{R}^p.$$

The encoding has only K weights. Decoding returns p coordinates, but the decoded point must lie in S. It equals x precisely when x already belongs to S.

Example 19: Two coordinates, one weight, two coordinates

Take $x = (2, 3)^\top$ and keep only the horizontal unit direction $c_1 = (1, 0)^\top$. With $C = c_1$, encoding gives

$$w = C^\top x = 1 \cdot 2 + 0 \cdot 3 = 2.$$

Decoding uses that weight to produce

$$\hat{x} = Cw = 2\begin{pmatrix}1\\0\end{pmatrix} = \begin{pmatrix}2\\0\end{pmatrix}.$$

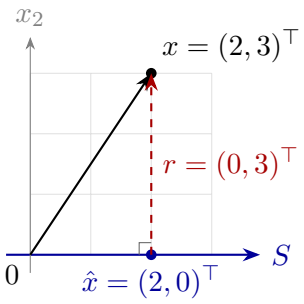
The process is

$$(2, 3)^\top \xrightarrow{\text{encode}} 2 \xrightarrow{\text{decode}} (2, 0)^\top.$$

The horizontal coordinate is kept. The vertical coordinate is lost: even though the decoded vector has two entries, its second entry must be zero for this choice of direction.

3.3 Measuring reconstruction error

To judge a reduced representation, we measure the part of each observation that decoding leaves out. For our example, this is the vertical segment from the projected point to the original point.



The residual is $r = x - \hat{x} = (0, 3)^\top$. Its length is 3, the perpendicular distance to S. The *squared* reconstruction error is $\|r\|_2^2 = 0^2 + 3^2 = 9$. For a plane in three dimensions, the same idea measures the perpendicular segment from a point to the plane.

Definition 3.3: Squared reconstruction error

For an observation $x^{(i)}$ and its reconstruction $\hat{x}^{(i)} = Cw^{(i)}$, the squared reconstruction error is

$$E_i = \|x^{(i)} - \hat{x}^{(i)}\|_2^2 = \sum_{j=1}^{p} \left(x_j^{(i)} - \hat{x}_j^{(i)}\right)^2.$$

It adds up the squared errors in the original p coordinates. It is zero when the observation is reconstructed exactly.

3.3.1 From one observation to the dataset

For a fixed C, choose the weights of each of the n observations to minimise the average squared error:

$$\min_{w^{(1)},\ldots,w^{(n)}} \frac{1}{n} \sum_{i=1}^{n} \|Cw^{(i)} - x^{(i)}\|_2^2.$$

With orthonormal columns, the best weights are $w^{(i)} = C^\top x^{(i)}$. Substituting them gives the error left for that choice of directions:

$$\boxed{g(C) = \frac{1}{n} \sum_{i=1}^{n} \|CC^\top x^{(i)} - x^{(i)}\|_2^2.}$$

For this fixed subspace, changing the weights cannot make the residuals shorter: the projections are already the closest points. Choosing a different subspace can change the error. This makes $g(C)$ a way to compare possible choices of directions.

3.4 Learning the basis from the data

So far, C was given and we found the weights. We now also want to learn C: which K directions give a useful representation of the whole dataset? The choice should depend on how the observations are arranged in space.

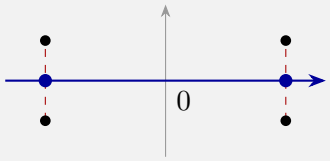
Example 20: Comparing two choices of direction

Consider four mean-centred observations:

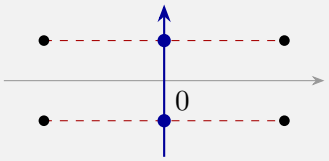
$$(-3,-1)^\top, \quad (-3,1)^\top, \quad (3,-1)^\top, \quad (3,1)^\top.$$

We want to keep only one weight per observation. The black dots are the same data in both panels; blue dots show their projections.

Keep the horizontal direction



Keep the vertical direction



The perpendicular segments are shorter for the horizontal line. Keeping $c_1 = (1,0)^\top$ removes only the vertical coordinate ± 1, so each squared error is 1 and their average is $g(C) = 1$. Keeping $c_1 = (0,1)^\top$ removes the horizontal coordinate ± 3, so each squared error is 9 and $g(C) = 9$.

Both choices use one weight. The horizontal direction reconstructs these observations better: it preserves their larger horizontal spread.

Definition 3.4: Principal Component Analysis

For mean-centred numerical data and a chosen dimension $K < p$, Principal Component Analysis (PCA) learns K perpendicular unit directions that minimise the average squared reconstruction error. The columns of C contain these directions; each observation is encoded by $w^{(i)} = C^\top x^{(i)}$ and reconstructed by $\hat{x}^{(i)} = Cw^{(i)}$.

PCA is *unsupervised*: it learns the directions from the input observations, without target labels. We specify how many directions to keep; the data determine which directions are useful. Mean-centering places the data cloud around the origin, where these subspaces pass.

The next question is how to find the directions that minimise $g(C)$.

Source: Spinelli, Lecture 03 (PCA), slides 2–8.